

EXPERIMENT 31

Firewall Log Investigation

AIM

To identify repeated blocked connection attempts and determine the targeted destination ports.

ALGORITHM

1. Read simulated firewall records.
2. Select blocked connections.
3. Group them by source IP and destination port.
4. Count each combination.
5. Report combinations with repeated attempts.

CODE

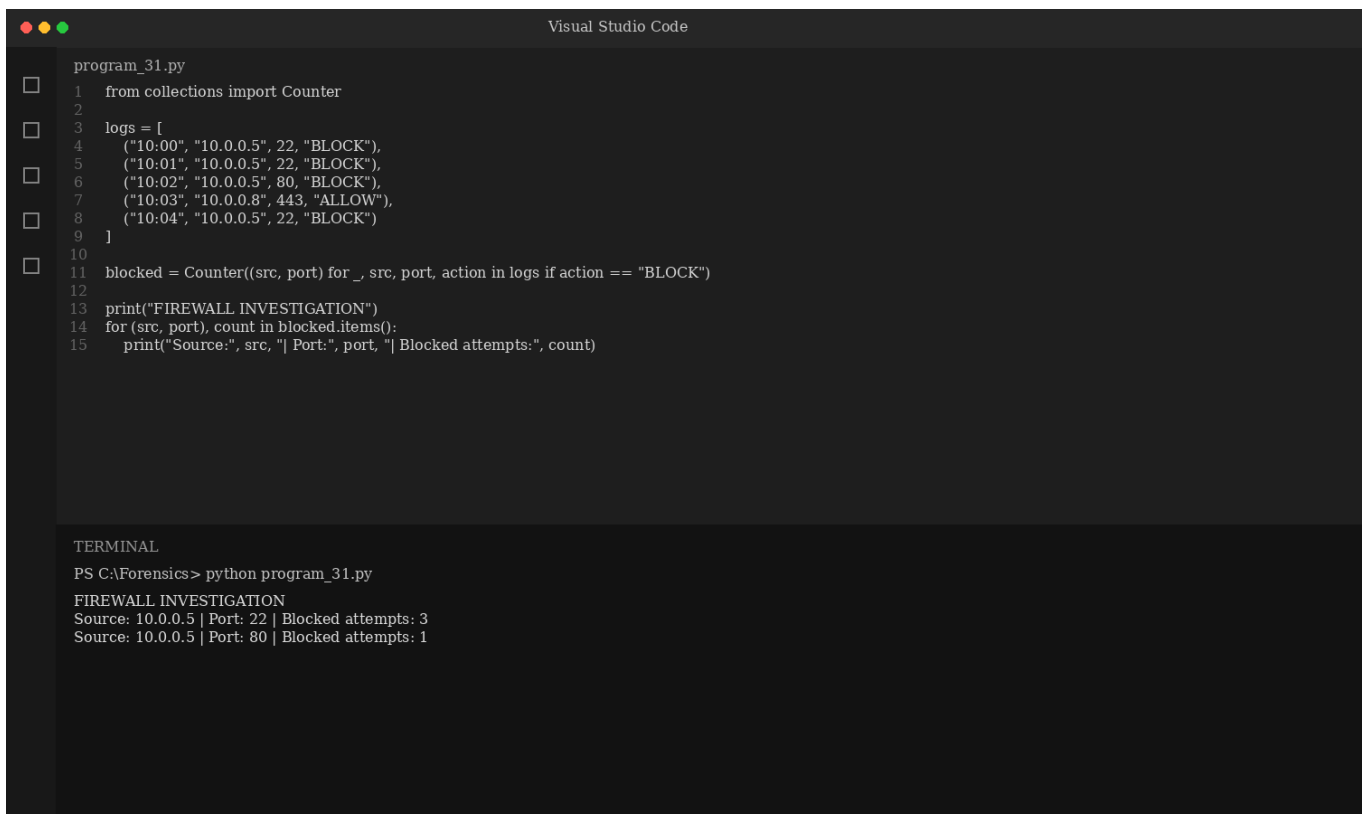
```
from collections import Counter

logs = [
    ("10:00", "10.0.0.5", 22, "BLOCK"),
    ("10:01", "10.0.0.5", 22, "BLOCK"),
    ("10:02", "10.0.0.5", 80, "BLOCK"),
    ("10:03", "10.0.0.8", 443, "ALLOW"),
    ("10:04", "10.0.0.5", 22, "BLOCK")
]

blocked = Counter((src, port) for _, src, port, action in logs if action == "BLOCK")

print("FIREWALL INVESTIGATION")
for (src, port), count in blocked.items():
    print("Source:", src, "| Port:", port, "| Blocked attempts:", count)
```

OUTPUT



The screenshot shows a Visual Studio Code editor window with a file named `program_31.py`. The code defines a list of firewall logs and uses a `Counter` to track blocked attempts by source IP and port. The terminal output shows the results of the investigation.

```
program_31.py
1  from collections import Counter
2
3  logs = [
4      ("10:00", "10.0.0.5", 22, "BLOCK"),
5      ("10:01", "10.0.0.5", 22, "BLOCK"),
6      ("10:02", "10.0.0.5", 80, "BLOCK"),
7      ("10:03", "10.0.0.8", 443, "ALLOW"),
8      ("10:04", "10.0.0.5", 22, "BLOCK")
9  ]
10
11 blocked = Counter((src, port) for _, src, port, action in logs if action == "BLOCK")
12
13 print("FIREWALL INVESTIGATION")
14 for (src, port), count in blocked.items():
15     print("Source:", src, "| Port:", port, "| Blocked attempts:", count)
```

TERMINAL

```
PS C:\Forensics> python program_31.py
FIREWALL INVESTIGATION
Source: 10.0.0.5 | Port: 22 | Blocked attempts: 3
Source: 10.0.0.5 | Port: 80 | Blocked attempts: 1
```

RESULT

The program counts blocked firewall attempts and identifies repeated source-port combinations.